Full-Stack / Backend Intern Technical Assessment (72 Hours)

Context

AIRMAN Academy (Skynet) helps Flight Training Organisations manage:

- Students
- Instructors
- Training progress
- Evaluations & safety culture

For this assignment, you will build a **mini Skynet EPR (Electronic Progress & Performance Record)** web application.

The goal is to see how you:

- Design data models
- Build REST APIs
- Build a usable web UI
- Think about performance tracking

This assessment has **two levels**:

- **Level 1 – Core EPR App (Required)**
- **Level 2 – Advanced EPR Features (Optional, Strong Plus)**

Completing Level 2 (even partially) will be a bonus during evaluation.

Tech Stack (Required)

We want you to use a stack close to AIRMAN's:

Backend

- **Node.js** with **TypeScript**
- Any framework: **Express**, **Fastify**, or **NestJS**
- **PostgreSQL** as the database
- **No Prisma**. Use:
  - Knex, Drizzle, TypeORM, or
  - Raw SQL + migration scripts

Frontend

- **React** or **Next.js** with TypeScript
- Any UI layer:
  - Tailwind / Chakra / MUI / simple CSS modules – your choice

Tooling & Repo

- GitHub repo (public or private with access)
- Clear **README**:
  - How to set up DB
  - How to run migrations / seed
  - How to start backend & frontend

AI Usage (Encouraged)

You are **encouraged to use AI tools** (ChatGPT, Copilot, Cursor, etc.) to:

- Scaffold boilerplate code
- Research patterns
- Generate documentation & comments
- Draft SQL queries, tests, etc.

But:

- You are responsible for understanding and reviewing the code.
- If you use AI heavily, add a short section in your README:

  > "How I used AI in this project (tools + what they helped with)"

We care about your judgment, not just speed.

LEVEL 1 – Core Skynet EPR (Required)

Goal

Build a **basic Skynet EPR web app** where:

- Admin can see a **list of people** (students & instructors)
- Admin can create & view **performance records** (EPRs) for each person
- There is a clean **web UI** to browse & inspect performance

Think of it like a mini "Performance Review + Progress Log" system for a flight school.

---

1. Data Model & Database (Level 1)

Design & implement PostgreSQL tables for:

1. `users`

- `id` (PK)
- `name`
- `email`
- `role` – enum/string: `"student" | "instructor" | "admin"`
- `created_at`, `updated_at`

2. `courses`

- `id`
- `name` – e.g. "CPL Integrated", "PPL"

- `license_type` – e.g. `"PPL"`, `"CPL"`, `"ATPL"`
- `total_required_hours` (numeric)

3. `enrollments`

- `id`
- `student_id` (FK → users.id)
- `course_id` (FK → courses.id)
- `start_date`
- `status` – `"active" | "completed" | "dropped"`

4. `epr_records` (Electronic Progress & Performance Records)

Each EPR is a structured review of a student or instructor.

- `id`
- `person_id` (FK → users.id) – the person being evaluated
- `evaluator_id` (FK → users.id) – who wrote the EPR (usually an instructor/admin)
- `role_type` – `"student"` or `"instructor"` (so we can differentiate)
- `period_start` (date)
- `period_end` (date)
- `overall_rating` (integer 1–5)
- `technical_skills_rating` (1–5)
- `non_technical_skills_rating` (1–5) // CRM, communication, checklist discipline, etc.
- `remarks` (text)
- `status` – `"draft" | "submitted" | "archived"`
- `created_at`, `updated_at`

Requirements

- Use a migration system compatible with your chosen DB tool (Knex, Drizzle, TypeORM, etc.)

- Add foreign keys with `ON DELETE RESTRICT` or `ON DELETE CASCADE` where sensible
- Indexes:
  - `epr_records.person_id`
  - `epr_records.evaluator_id`
  - `epr_records.period_start`, `period_end`

Seed data:

- 2–3 instructors
- 6–10 students
- 1–2 courses
- Enrollments linking students to courses
- A handful of EPR records for a couple of students & instructors

---

2. Backend APIs (Level 1)

Implement REST APIs for:

A. Directory – People List

GET `/api/people`

- Query params:
  - `role` (optional: `"student"` or `"instructor"`)
  - `search` (optional: name/email substring)
- Returns a list of:
  - `id`, `name`, `email`, `role`
  - For students:
    - `course_name`
    - `status` (from enrollment)
  - For instructors:

- `total_eprs_written` (count of `epr_records` as evaluator)

B. EPR List & Detail

GET `/api/epr`

- Query params:
  - `personId` (required)
- Returns all EPR records for that person ordered by `period_start DESC`.

GET `/api/epr/:id`

- Returns full details of a single EPR record for viewing.

C. Create & Update EPR

POST `/api/epr`

- Body:
  - `personId`
  - `evaluatorId`
  - `roleType`
  - `periodStart`
  - `periodEnd`
  - `overallRating`
  - `technicalSkillsRating`
  - `nonTechnicalSkillsRating`
  - `remarks`
  - `status`
- Validate:
  - Ratings are between 1 and 5
  - `periodEnd >= periodStart`

- `personId` and `evaluatorId` exist

PATCH `/api/epr/:id`

- Allow updating:
  - ratings
  - remarks
  - status (e.g. move from `draft` to `submitted`)

Authentication can be **simplified** for this task:

- You can hardcode a single "logged-in admin" user id or accept a fake `x-user-id` header, or just skip full auth – but keep the API shape clean.

---

3. Frontend – Skynet EPR UI (Level 1)

Build a simple, clean **single-page web app** with **two main screens**:

3.1 People Directory Screen

- Left pane: Filters + list
- Right pane: Selected person details

Left pane:

- Tabs or toggle buttons:
  - **Students | Instructors**
- Search input:
  - Filters by name/email using `/api/people?search=...`
- List:
  - Display:
    - Name
    - Role badge
    - For students: course & enrollment status
    - For instructors: count of EPRs written

Right pane (when a person is selected):

- Show:
  - Name
  - Role
  - Course (if student)
- "Performance Records" section:
  - List EPRs from `/api/epr?personId=...`:
    - Period (e.g. `Jan–Mar 2025`)
    - Overall rating
    - Status badge (`draft`, `submitted`)
  - Clicking an EPR opens the **EPR detail view** (see below).

3.2 EPR Detail & Editor

When you click an EPR:

- Show a **detail panel** or modal with:
  - Person name
  - Period start/end
  - Overall rating + per-category ratings
  - Remarks text
- Add **edit mode** (can be inline or separate "Edit" screen):
  - Form with:
    - 3 rating inputs (overall, technical, non-technical)
    - Remarks textarea
    - Status dropdown
  - Save → calls `PATCH /api/epr/:id`

Add a **"New EPR"** button on the person details pane:

- Opens a simple form with fields for:
  - period start/end
  - ratings
  - remarks
- On submit → `POST /api/epr` and refresh list.

> UI doesn't need to be super fancy. The key is: clean layout, sensible state management, and correct API usage.

LEVEL 2 – Advanced Skynet EPR Features (Optional)

For candidates who want to stand out, extend your Skynet EPR app with **any/all** of the following.

Even **one or two** of these features, done well, is a big positive.

Level 2 – Option A: Progress Summary & Analytics

Add an endpoint:

GET `/api/epr/summary/:personId`

Return:

```
{
  "personId": "uuid",
  "roleType": "student",
  "averageOverallRating": 4.3,
  "averageTechnicalRating": 4.1,
  "averageNonTechnicalRating": 4.5,
  "eprCount": 5,
  "lastThreePeriods": [
    { "periodLabel": "Q1 2025", "overallRating": 4.2 },
    { "periodLabel": "Q4 2024", "overallRating": 4.0 },
```

```
    { "periodLabel": "Q3 2024", "overallRating": 4.5 }
  ]
}
```

On the frontend:

- In the person's right pane:
  - Show a **"Performance Snapshot"** card:
    - Average overall rating
    - Small badges for technical & non-technical averages
    - A simple "trend list" of last 3 periods

Level 2 – Option B: Instructor vs Student Modes (Mini Role-Based UX)

Implement a simple role UX:

- If the "logged in" user (hardcode or query param) is:
  - `instructor` → They can:
    - Create **student EPRs** where they are `evaluatorId`
    - View student EPRs they wrote
  - `student` → They can only:
    - View **their own EPRs** (read-only)

Implementation idea:

- Pass `currentUserRole` and `currentUserId` as mock "session" in frontend.
- Add basic backend checks like:
  - If `role = student`, and `personId != currentUserId` → 403.

No need for JWT or full auth; just show you understand **access control**.

Level 2 – Option C: Simple “AI Assistant” Stub for EPR Drafts

Create a fake “AI assist” endpoint:

POST `/api/epr/assist`

- Body:
  - `overallRating`, `technicalSkillsRating`, `nonTechnicalSkillsRating`
- Returns:

```
{
  "suggestedRemarks": "The student demonstrates strong technical funda
mentals but should focus on..."
}
```

Implementation:

- **No real LLM needed**.
- Use simple rule-based logic:
  - If scores are high, generate positive remarks.
  - If some are low, generate a “needs improvement” remark.

On the UI:

- Add a “Generate Suggested Remarks” button in the EPR form:
  - Calls `/api/epr/assist`
  - Fills the remarks textarea

This aligns nicely with how AIRMAN actually uses AI in Skynet.

Evaluation Criteria

We will evaluate your submission on:

1. **Architecture & Code Quality (30%)**
   - Clear separation: routes / controllers / services / DB

- Consistent TypeScript usage, types/interfaces

2. **Data Modeling & SQL (25%)**
   - Good relational design
   - Foreign keys & indexes
   - Sensible use of enums/strings for status, roles
3. **Frontend UX & State Management (25%)**
   - Clean, readable screens
   - Directory + EPR interactions feel logical
   - Handles loading and error states
4. **Documentation & Setup (10%)**
   - Clear README
   - Simple commands to run backend + frontend
5. **Level 2 Bonus (10%)**
   - Any advanced feature(s) that show deeper thinking:
     - Progress summaries
     - Basic role-based UX
     - AI-assist stub

What Candidates Should Submit

- GitHub repository link
- README including:
  - What is implemented (Level 1 only / Level 1 + which Level 2 options)
  - How to run:
    - `npm install` / `pnpm install`
    - DB setup + migrations
    - seed command

        - `npm run dev` / `pnpm dev` for FE/BE
    - Short section: “How I used AI in this project”